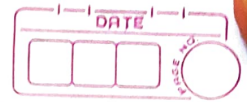


$$P \subseteq NP$$



DAA unit 5

Complexity Theory & Parallel Algorithm

1. Differentiate betⁿ deterministic and non-deterministic algo with example.
2. What are steps to prove NP-completeness of a problem? Prove that vertex cover problem is NP-complete.
3. Explain what is NP-Hard problem and justify why TSP is NP-Hard.
4. Prove that NP complete problem - Vertex cover problem
5. What is COOK's Theorem?
6. Show that job seq with deadline problem is NP hard.
7. What are the conditions to prove that a problem P is NP-complete?
8. Show an ex. where SATISFIABILITY is reduced to VERTEX COVER.

(1) Diff.

Deterministic Algo.

- ① Follows a predictable seq of steps for any given input

- ② Always produce the same output for a particular input

- ⑤ Examples include linear search, binary search, bubble sort.

- ④ These algo. are easier to understand and analyze because they exhibit consistent behaviour.

- ⑤ Deterministic algos execute in straight-forward manner.

- ⑥ Det. algo provides certainty about their outcome

Non-Deterministic Algo

- ① May exhibit different behaviours for same input

- ② Produces a range of possible outputs for a given input

- ③ Examples include randomized algo like quicksort & genetic algo.

- ④ Their behaviour can depend on factors such as random no. generation or specific state of system.

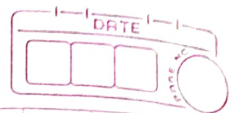
- ⑤ Non-cler. algs execute in random manner

- ⑧ Non-definite algo introduces uncertainty.

Example :-

- Deterministic Algo: Binary Search.
Given a sorted array, binary search always follows the same steps to find the target element. It divides the array in half repeatedly until it finds the target.
- Non-deterministic algo: Randomized Quicksort. Quicksort, when implemented with a random pivot selection, is non-deterministic because the choice of pivot affects the partitioning of the array, and diffⁿ runs with same input may produce diffⁿ outcomes.

NP $\rightarrow$ nondeterministic polynomial time.



(2) NP-completeness

(a) Proving NP-completeness typically involves two main steps:

① Showing the problem is in NP:
This means you can verify a solⁿ in polynomial time. Usually, this involves providing a polynomial-time algo. to verify solⁿ to the problem.

② Reducing a known NP-complete problem to the given problem:

This step demonstrates that the given problem is at least as hard as the known NP-complete problem. The reduction should be polynomial-time computable.

Prove that Vertex Cover problem is NP-complete.

① To prove

We need to demonstrate two things:

Step 1: Vertex Cover is in NP.

• ~~Def~~ Vertex Cover defⁿ: A set $C \subseteq V$ such that every edge in E has at least one endpoint in C .

• Verification: Check each edge in E to ensure one endpoint is in C .

• Complexity: Verification takes $O(|E|)$ time, thus Vertex Cover is in NP.

Step 2: Vertex Cover is NP-Hard.

- Reduction From 3-SAT:

- 3-SAT Problem: Determine if a boolean formula is 3-CNF is satisfiable.

- Construction:

1. For each variable x_i , create vertices x_i and $-x_i$, with an edge between them

2. For each clause C_i , create a triangle of vertices rep. the literals.

3. Connect literals in clauses to corresponding variable vertices.

- Vertex Cover Size k : set $k = n + 2m$.
(n - no. of variables, m - no. of clauses).

- Correctness:

- IF ϕ is satisfiable: choose true literals & approximate vertices from each clause triangle to form a vertex cover of size $n + 2m$.

- IF a vertex cover ^{of} size k exists:

Ensures the correct truth assignment by covering edges consistently.

Conclusion:

Since Vertex Cover is in NP and is NP-hard we conclude that the vertex cover problem is NP-complete.

DATE

Q. Explain what is NP-hard problem. and Justify why TSP is NP-hard.

① An NP-hard problem is a class of problem for which no efficient soln algorithm is known. However, if a soln to one NP-hard problem could be found quickly, then every problem in NP could be solved quickly.

② The traveling Salesman Problem (TSP) is NP-hard because finding the shortest route that visits each city exactly once is computationally infeasible to solve efficiently as the no. of cities grows. There is no known algo. that can solve all instances of TSP efficiently, which is why it is classified as NP-hard.

Q. What are the conditions to prove that a problem P is NP complete:

→ To ' , two conditions must be satisfied:

① P is in NP: The problem P must be verified in polynomial time. This means that given a soln, one can verify its correctness within polynomial time.

② NP-Hardness: Every problem in NP can be reduced to P in polynomial time. This means there exists a polynomial time reduction from any NP problem to P . If one could solve P in poly. time, then every problem in NP could also be solved in poly. time.